SE

Lab 5: Static Code Analysis

Name : Sahana Shivakumar Badiger

SRN : PES2UG24CS427

Section : G

Issue Table

Issue	Type	Line(s)	Description	Fix Approach
Shell command with `os.system`	Security (High)	63	Bandit B605: command built from the `filename` variable, so command injection is possible	Removed `os.system` and used `print()` instead
Weak MD5 hash	Security (High)	52	Bandit B324: MD5 is insecure for hashing	Replaced with `hashlib.sha256`
Use of `eval()`	Security (Medium)	47	Bandit B307, Pylint W0123: runs arbitrary code from input. Pylint W0612 and Flake8 F841 also flag `base_fee` (line 46) as unused	Replaced the expression string with a numeric `discount_rate`, validated between 0 and 1
Hardcoded secret token	Security (Low)	75	Bandit B105: secret stored in source code (and printed)	Read it from the `REGISTRAR_AUTH_TOKEN` environment variable and stopped printing it
Insecure random number generator	Security (Low)	56	Bandit B311: `random` is not suitable for OTPs	Used `secrets.randbelow`

Issue	Type	Line(s)	Description	Fix Approach
Bare `except` with `pass`	Bug (Low)	31-32	Bandit B110, Pylint W0702, Flake8 E722: catches everything and hides errors	Catch only `KeyError` and return `False`
Mutable default argument	Bug	15	Pylint W0102: `audit_trail=[]` is shared across calls	Default to None and create the list inside the function
`open()` without `with` or encoding	Bug (resource)	60	Pylint R1732, W1514: file may not close, no encoding given	`with open(filename, "w", encoding="utf-8")`
Global statement and constant naming	Design	12, 16	Pylint W0603, C0103: `global total_tuition` and a lowercase constant name	Kept state in a `STATE` dict and renamed `courses` to `COURSES`
Unused import `math`	Style	4	Flake8 F401, Pylint W0611	Removed the import
Unused variable `lab_fee`	Style	36	Flake8 F841, Pylint W0612	Removed the variable
Comparison `== None`	Style	37	Flake8 E711, Pylint C0121	Used `.get()` with `is not None`
Unnecessary `else` / simplifiable if	Style	39	Pylint R1703, R1705	Returned the boolean expression directly
Old-style `%` formatting	Style	23	Pylint C0209	Used an f-string
camelCase function names	Style	15, 27, 35, 45, 51, 55, 59, 66	Pylint C0103: not snake_case	Renamed to `enroll_student`, `drop_course`, `check_course_status`, etc.
Redefined outer name `courses`	Design	66	Pylint W0621: parameter shadows the global	Renamed the parameter to `course_map`

Issue	Type	Line(s)	Description	Fix Approach
Unused loop variable `cid`	Style	68	Pylint W0612	Replaced the loop with `sum(...)` over `.values()`
Lines too long	Style	7, 8, 9, 15, 23, 79	Flake8 E501, Pylint C0301	Wrapped lines to 79 characters or fewer

Summary (Before vs After)

Tool	Before	After
Pylint	4.00/10 (34 messages)	9.66/10 (2 messages)
Bandit	6 issues (2 High, 1 Medium, 3 Low)	0 issues
Flake8	12 issues	1 issue

reflection questions:

1.Which issues were the easiest to fix, and which were the hardest? Why?
The easiest were the style issues: removing the unused math import and the unused lab_fee variable, changing == None to is None, switching to an f-string, and wrapping long lines. Each report gave the exact line and the fix was mechanical. The hardest was the eval() in calculateScholarshipTuition (Bandit B307). It couldn't be fixed with a one-line change, because I had to redesign the function to take a numeric discount_rate, validate it and update the caller. The hardcoded token and the global total_tuition were also harder, because they needed design decisions (environment variables and a state dictionary) rather than simple edits.

2.Did the static analysis tools report any false positives? If so, describe one example.
Yes. Pylint (W0612) and Flake8 (F841) reported base_fee at line 46 as an unused variable. In the original code it is actually used, because the string passed to eval() ("base_fee * 0.75") refers to it. The tools can't see variable names inside a string, so they flagged it. This shows that reports need human review before every warning is acted on. Removing that variable blindly would have broken the original program.

3.How would you integrate static analysis tools into your actual software development workflow?
I would run Flake8 and Pylint in my editor so problems show up while I write the code. I would add a pre-commit hook that runs all three tools before each commit. In CI (for example GitHub Actions), I would run them on every pull request and fail the build on High or Medium Bandit findings or on a Pylint score below an agreed threshold. I would keep shared settings in config files (.pylintrc, setup.cfg) so the whole team uses the same rules. Confirmed false positives would be suppressed individually (with # nosec or # pylint: disable=...) along with a comment explaining why.

4.What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?
The numbers improved a lot: Pylint went from 4.00/10 to 9.66/10, Bandit from 6 issues (2 High, 1 Medium, 3 Low) to 0, and Flake8 from 12 issues to 1. The code is more readable thanks to snake_case names, docstrings, f-strings and shorter lines. It is more robust because the bare except now catches only KeyError, inputs are validated, the shared mutable default argument is gone, and files are opened with with. It is also more secure: the shell injection risk, eval, weak MD5 hashing, insecure random OTPs and the hardcoded secret are all removed. The program still produces the same output.